VIEWFLIX - Movie & Series Streaming Platform Database

Database Management Systems (COMP2004) - Term Project

Team: Goktug Karaca, Defne Kaya Date: 29 May 2026

1. System and Requirements

VIEWFLIX is the database for a movie and series streaming service like Netflix. Users sign up with their own details, log in, and watch films and series. They can mark favorites, keep a watch history, and rate or review titles. We only built the database layer; there is no web or mobile app. Main requirements:

- Users register with personal info (email, name, country, birth date) and log in.
- Users pick one of three plans (Basic, Standard, Premium) and the payment status is tracked.
- Content is either a movie or a series.
- Each title has one or more genres and a cast of actors and directors.
- Users keep favorites, a watch history, and reviews with a rating.

2. Logical Design (E-R Model)

The system has 11 related tables. All tables, keys and relationships are defined in schema.dbml and drawn as an E-R diagram on dbdiagram.io. Main relationships:

Relationship	Cardinality	Note
users - subscriptions - subscription_plans	1 : N : 1	A user has a subscription on a plan
users - reviews / favorites / watch_history	1 : N	User activity
content - reviews / favorites / watch_history	1 : N	Activity on a title
content - genres	N : M	via content_genres (junction table)
content - people	N : M	via content_cast (junction table, with role)

2.1. Normalization (1NF to 3NF)

To show the design avoids redundancy, here is how a single flat table would be broken down. Start with one wide table:

Watch(user_email, user_name, plan_name, plan_price, title, genres, director, rating)

- **Anomalies:** Insertion - cannot add a plan or a title until someone uses it. Update - changing a plan price means editing every matching row. Deletion - removing the last view of a title loses the title itself.
- **Functional dependencies:** user_email -> user_name; plan_name -> plan_price; content_id -> title, director.
- **1NF:** genres is multi-valued, so it moves to its own rows: content_genres(content_id, genre_id).
- **2NF:** Partial dependencies are removed by splitting out users, subscription_plans and content.
- **3NF:** Transitive dependencies are removed: plan_price (which depends on plan_name, not the key) goes to subscription_plans; director goes to people + content_cast.

All 11 tables end up in 3NF: every non-key column depends fully and directly on its primary key. Single attributes such as country and original_language have no further dependency, so they stay as plain columns. The decomposition is lossless and dependency preserving.

3. Physical Design

01_create_tables.sql builds every table on MySQL 8.0. Constraints used:

- **Primary keys:** on every table; composite keys on the junction tables content_genres, content_cast and favorites.
- **Foreign keys:** on all relationships, with ON DELETE CASCADE where it makes sense (deleting a user also removes their subscription, reviews, favorites and history).
- **UNIQUE:** users.email, genres.genre_name, and reviews(user_id, content_id) so a user reviews a title once.
- **NOT NULL / CHECK:** rating BETWEEN 1 AND 10, progress_percent BETWEEN 0 AND 100, content_type IN ('Movie','Series'), payment_status IN ('Success','Failed','Refunded'), and similar.

Sample data: 02_sample_data.sql loads about 228 rows across the 11 tables (15 users, 20 titles, 20 people, 32 cast rows, 52 genre links, 25 history rows, 20 reviews, 16 favorites). All rows load cleanly with foreign key checks on.

4. SQL Implementation

Every statement has a short business question above it.

- **03_dml_operations.sql:** 14 statements - 5 INSERT, 5 UPDATE, 4 DELETE (new signup, subscription, favorite, review, plan downgrade, expiring subscriptions, account deletion with cascade, and more).
- **04_queries.sql:** 5 simple queries (filtering and sorting) and 7 complex ones using JOIN, GROUP BY / HAVING and subqueries: average rating per title, busiest genres, monthly revenue per plan, titles above the average rating, titles never watched, top 5 favorites, and titles by a given director.

Testing and demo: All files (schema, data, DML and the 12 queries) were run on a real MySQL 8.4 server with no errors. The live demo runs in MySQL Workbench in order 01 to 04; the E-R diagram is shown on dbdiagram.io from schema.dbml.